



# Lecture 1: What is Svelte?

## INTERVIEW QUESTION

[> CORE](#)

*What is Svelte, and how does it differ from frameworks like React and Vue?*

You've heard Svelte "has no virtual DOM" — but what does that actually mean, and why does it matter for performance?

When building web applications, developers want to write declarative code (saying *what* the UI should look like based on state) instead of imperative code (writing out the exact steps to update the DOM). Frameworks like React and Vue achieve this by doing heavy work in the browser while the user is using the app. They load a large "runtime" engine, keep a memory representation of the entire UI (the Virtual DOM), and constantly compare the old state to the new state to figure out what changed.

Svelte takes a fundamentally different approach. It is not just a library you include in your webpage; it is a **compiler**.

---

## The Compiler Advantage

Instead of doing the work in the browser, Svelte does the heavy lifting on your computer during the build step. It analyzes your `.svelte` files (which contain HTML, CSS, and JavaScript) and compiles them into highly optimized, standard JavaScript before the code is ever sent to the user.

### KEY TAKEAWAY

What the DOM is to HTML, the AST is to code.

Just as the browser parses a flat HTML string into a Document Object Model (DOM) tree so it can manipulate the web page, a compiler parses a flat string of code into an Abstract Syntax Tree (AST) so it can analyze and manipulate the program. Svelte reads your declarative components, builds an AST, and figures out exactly how the state connects to the DOM.

Because Svelte knows exactly what can change at compile time, it generates vanilla JavaScript that directly targets those specific DOM nodes.



App.svelte

```
1  <script>
2    let count = $state(0);
3
4    function increment() {
5      count += 1;
6    }
7  </script>
8
9  <button onclick={increment}>
10    Clicks: {count}
11  </button>
```

*Directly updates the* **TEXT** *node, no diffing required*

---

## No Virtual DOM

### INTERVIEW TIP

**To impress the interviewer:** During an interview, most candidates will simply state that "Svelte doesn't use a Virtual DOM." If you want to show deep technical background, explain *why* it doesn't need one — this proves you understand the mechanics, not just the marketing.

React and Vue need a Virtual DOM because their runtime doesn't know what changed until it diffs the old virtual tree against the new one. Svelte doesn't need a Virtual DOM because its compiler has already wired the variable `count` directly to the specific text node inside the `<button>`. When `count` changes, Svelte's generated code surgically updates that exact spot.

By compiling away the framework, Svelte delivers smaller bundle sizes, faster initial load times, and eliminates the runtime overhead of diffing trees in memory.

---

## Summary

Svelte shifts the heavy lifting of building user interfaces from the browser (runtime) to your local machine (compile time). It takes declarative components and compiles them into highly optimized JavaScript that updates the DOM surgically, without needing an intermediate Virtual DOM.

|                      | <b>REACT / VUE</b><br><b>(Runtime Library)</b> | <b>SVELTE</b><br><b>(Build-Time Compiler)</b>   |
|----------------------|------------------------------------------------|-------------------------------------------------|
| <b>Architecture</b>  | Ships a framework engine to the browser        | Compiles components to vanilla JS at build time |
| <b>State Updates</b> | Virtual DOM diffing                            | Direct, surgical DOM updates                    |
| <b>Bundle Size</b>   | Includes the framework runtime engine          | Contains only the generated vanilla JavaScript  |

|                    | <b>REACT / VUE</b><br><b>(Runtime Library)</b> | <b>SVELTE</b><br><b>(Build-Time Compiler)</b>     |
|--------------------|------------------------------------------------|---------------------------------------------------|
| <b>Performance</b> | Overhead from keeping a virtual tree in memory | Bare-metal performance by updating nodes directly |